

Week 4 Interview Preparation

Observability, Performance & Auto-Healing

Objective: Build a complete observability stack → understand performance bottlenecks → build auto-healing & scaling systems.



Section 1: Prometheus Fundamentals

Question 1: Explain how Prometheus scrapes metrics. What is the role of service discovery in Kubernetes?

How to Answer

Use a logical flow:

1. Pull model
2. /metrics endpoint
3. Service discovery
4. Relabeling
5. Scrape config

What the Interviewer Wants: They want to know if you understand that Prometheus *does not push* and that Kubernetes SD is dynamic.

What NOT to Do

- Don't say "Prometheus listens for metrics". Incorrect.
- Don't jump directly into YAML without conceptual clarity.

Sample Answer

Prometheus works on a pull model. Every target exposes a /metrics endpoint, and Prometheus periodically scrapes it.

In Kubernetes, Prometheus uses the API server for service discovery. Instead of manually adding IPs, it discovers Pods, Services, Endpoints automatically through labels. Relabeling rules ensure that only valid targets are scraped.

This model makes Prometheus fault-tolerant because losing a pod does not break the configuration; SD updates the target list dynamically.

Question 2: Counters vs gauges vs histograms vs summaries. Explain with examples.

How to Answer: Define → Use case → Example → Pitfalls.

What the Interviewer Wants: Clarity on metric types and when to use them.

What NOT to Do

- Do not mix up histograms and summaries.
- Do not say “histograms store raw data”. They store buckets, counts, sums.

Sample Answer

Counters always increase; they’re good for errors, requests, restarts.

Gauges go up and down, such as memory usage or queue depth.

Histograms bucket observations and allow quantiles like p90 latency. Widely used in APIs.

Summaries calculate quantiles locally at the client SDK level but cannot be aggregated reliably across instances.

In production, histograms are preferred for distributed systems because aggregation is possible at Prometheus level.

Question 3: How do you detect high cardinality and why is it dangerous?

How to Answer: Definition → Cause → Impact → Detection.

What NOT to Do

- Do not blame Prometheus “limits”.
- Do not say cardinality is bad always; only uncontrolled cardinality is harmful.

Sample Answer

High cardinality happens when a metric generates too many unique label combinations.

Common causes include per-request labels, user IDs, session IDs, or timestamps. It increases memory usage, slows down queries, and can crash Prometheus.

You detect it using metrics like: `prometheus_tsdb_head_series` or `topk` queries on label count.

Fixing involves removing unstable labels, adding relabel drops, and tightening instrumentation.

Section 2: PromQL Scenarios

Question 4: `rate()` vs `irate()`: when do you use which?

How to Answer: Definition → Behavior → Use case → Anti-pattern.

Sample Answer

`rate()` uses a range window and gives a smooth rate over time. It's used for dashboards and SLO calculations.

`irate()` uses only the last two samples. It reflects instant spikes.

Use `irate()` for alerting on sudden spikes (such as error burst) and `rate()` for long-term trends.

Never use `irate()` on noisy high-frequency data for dashboards; it misleads.

Section 3: Grafana & Dashboards

Question 5: What are best practices when building a production dashboard?

How to Answer: Show you understand signal vs noise.

What Interviewers Want

- Reduction of cognitive load
- Symptom-first design
- SRE principles

Sample Answer

Good dashboards follow the RED or USE monitoring model. They focus on latency, traffic, errors, saturation. Panels must be predictable: same colors mean same things. Avoid overloading dashboards with 20+ graphs; operators need fast triage. Every panel should answer one question clearly. Dashboards should default to meaningful time ranges and include annotations for deploys.

Section 4: Logging: Loki & FluentBit

Question 6: Why is label cardinality a bigger problem in Loki than Elasticsearch?

How to Answer: Contrast architecture → Explain chunk storage.

Sample Answer

Loki indexes only labels, not log bodies. This makes label choices extremely important. If a label generates too many values, Loki creates tons of index entries, leading to slow queries and huge storage use.

Elasticsearch distributes data differently and can tolerate higher cardinality. Loki cannot. The fix: avoid dynamic labels and rely on log body filters instead.

Question 7: Logs are missing in Loki. How do you troubleshoot?

How to Answer

Show an ordered debugging thought process:

1. Check if application logs exist locally (kubectl logs).
2. Check FluentBit input tail position.
3. Check FluentBit output plugin errors.
4. Check Loki distributor and ingester logs.
5. Check whether log labels exceed retention or query scope.

Sample Answer

First verify that logs exist on the container. Then check FluentBit's tail plugin to confirm it is reading the file and not stuck due to inode rotation.

Next inspect FluentBit output metrics and Loki distributor logs for rejections.

Finally ensure the query matches the correct labels and namespace. This covers the end-to-end path.

Section 5: Distributed Tracing & OpenTelemetry

Question 8: Explain spans, trace IDs, context propagation, and how missing spans occur.

How to Answer Define → Connect concepts → Give scenario.

Sample Answer

A trace represents a request path across services. Each unit of work is a span with duration and metadata. Trace IDs stay constant across all spans.

Context propagation ensures the trace ID travels through HTTP headers or gRPC metadata. Missing spans occur when a service drops or fails to forward the context, uses a different tracer, or sampling is misconfigured.

Section 6: Autoscaling (HPA, VPA, KEDA)

Question 9: Why would an HPA not scale even when CPU is high?

How to Answer Show understanding of metric pipeline.

Sample Answer

The most common reasons are:

- HPA using metrics.k8s.io but the Metrics Server is down.
- Prometheus adapter not publishing custom metrics.
- Pods using requests too high or too low.
- Insufficient node capacity preventing scheduling.
- Cooldown windows or stabilization windows delaying scaling.

I would check HPA events, metrics API, and the cluster autoscaler logs.

Question 10: What happens when both HPA and VPA work on the same deployment?

Sample Answer

It causes feedback loops. HPA reacts to load by scaling horizontally. VPA changes requests, which changes how HPA calculates utilization.

Kubernetes disallows full mode VPA with HPA to avoid conflict. In production, use HPA + VPA in recommendation mode only.

Section 7: SLOs, Alerts, SRE Concepts

Question 11: What is an error budget and why is it important?

How to Answer: Show understanding of operational decision-making.

Sample Answer

An SLO defines the allowed level of unreliability. The error budget is $1 - \text{SLO}$. If SLO is 99.9, your error budget is 0.1 percent downtime. It guides engineering choices.

If the error budget is nearly exhausted, teams freeze deploys and investigate issues. If there is excess, teams can release more aggressively. It aligns reliability with business needs instead of guessing.

THE DEVOPS WAR ROOM

Question 12: How do you design alerts that do not cause noise?

How to Answer: Symptoms, not causes → Multi-burn → Avoid flapping.

Sample Answer

Alert on user impact, not system internals. Use multi window, multi burn rate alerts to catch fast and slow burn patterns. Avoid instant metrics; use stable ranges such as 5m or 1h. Introduce grouping and routing in Alertmanager.

Every alert should answer:

- Is something broken
- Does someone need to act
- Is it actionable

If not, it should not be an alert.

How to Answer

- Think aloud
- Show debugging order
- Relate answers to real outages
- Explain tradeoffs
- Keep answers structured: Concept → Why → When → Example

What NOT to Do

- Do not jump to tools before fundamentals
- Do not say “it depends” without explaining what it depends on
- Do not define terms without connecting them to real scenarios





THE DEVOPS WAR ROOM